Projeto da Disciplina

Plataforma de Gestão de Demandas Urbanas (Smart City)

1. Visão geral do projeto

Ao longo da disciplina de **Desenvolvimento Web**, **trabalharemos em um projeto aplicac dividido em duas grandes etapas de entrega**, acompanhando a evolução de um produt digital do **frontend até o backend integrado**.

O projeto da turma será o desenvolvimento de uma **Plataforma de Gestão de Demanda Urbanas (Smart City)**.

A proposta busca simular um contexto real de desenvolvimento de software, no qual uma equipe precisa:

- Compreender um problema real;
- Projetar uma solução digital;
- Construir interfaces;
- Implementar regras de negócio;
- Persistir dados;
- Integrar frontend e backend.

A disciplina foi estruturada para que possamos **construir progressivamente uma aplica full stack**, utilizando tecnologias adotadas no mercado.

2. Tema do desafio

Tema principal do projeto

Plataforma de Gestão de Demandas Urbanas (Smart City)

Área / Domínio

Governo / Setor Público / Cidades Inteligentes

3. Contexto do problema

Cidades lidam diariamente com uma grande quantidade de solicitações relacionadas a serviços urbanos, como:

- Manutenção de vias
- Iluminação pública
- Coleta de lixo
- Saneamento
- Fiscalização
- Zeladoria urbana

Essas demandas costumam chegar por diversos canais, como:

- Telefone
- Atendimento presencial
- Redes sociais
- Aplicativos isolados
- Sistemas administrativos

Essa fragmentação dificulta:

- A organização das solicitações;
- O acompanhamento das demandas;

- A priorização de atendimentos;
- A geração de indicadores de gestão;
- A transparência para a população.

---

4. Problema central

A ausência de uma plataforma centralizada para **registro, rastreabilidade e priorização demandas urbanas** dificulta a tomada de decisão por parte da gestão pública e reduz a eficiência no atendimento à população.

---

5. Alinhamento com políticas públicas

A proposta dialoga com diretrizes brasileiras de transformação digital no setor públi

Entre elas está a **Lei nº 14.129/2021 — Lei do Governo Digital**, que estabelece princípios como:

- Eficiência administrativa;
- Transparência;
- Desburocratização;
- so de soluções digitais para melhorar serviços públicos.

A proposta também se relaciona com a **Estratégia Nacional de Governo Digital**, que incentiva o uso de tecnologias digitais para melhorar a relação entre governo e cidadão.

Conheça a Estratégia Nacional de Governo Digital: Governança Digital

---

6. Objetivo da solução

Neste projeto, construiremos uma aplicação web capaz de:

- Centralizar o registro de demandas urbanas;
- Permitir o acompanhamento das solicitações;
- Oferecer visibilidade para gestores públicos;
- Aumentar a transparência no atendimento à população.

7. Principais usuários da solução

Cidadão

Usuário responsável por:

- Registrar demandas urbanas;
- Acompanhar o status das solicitações.

Exemplos de demandas incluem:

- Buraco em via pública;
- Poste sem iluminação;
- Descarte irregular de lixo;
- Problemas relacionados a saneamento.

Gestor público

Usuário responsável por:

- Visualizar demandas registradas;
- Filtrar solicitações;
- Atualizar o status das demandas;
- Acompanhar indicadores operacionais.

8. Funcionalidades mínimas esperadas (MVP)

Ao longo do projeto, implementaremos no mínimo as seguintes funcionalidades:

1. Cadastro de usuários;
2. Autenticação de usuários;
3. Registro de demandas urbanas;
4. Definição de categoria da demanda;
5. Descrição da solicitação;
6. Registro de localização;
7. Acompanhamento do status da demanda;
8. Painel administrativo para gestores;
9. Filtros por tipo, região e status;
10. Visualização de métricas básicas.

Exemplos de métricas:

- Quantidade de demandas registradas;
- Tempo médio de atendimento;
- Distribuição de demandas por categoria.

---

9. Limites do escopo

Este projeto **não contempla**:

- Integração com sistemas reais de prefeituras ou governos;
- Execução real de serviços urbanos;
- Validade jurídica das solicitações registradas.

O foco da disciplina será **o desenvolvimento técnico da solução digital (Web - full stack)**.

---

10. Organização das entregas

A disciplina será dividida em **duas grandes etapas de entrega**, que acompanharão a evolução da aplicação.

Avaliação 1

Desenvolvimento do Frontend

Na primeira etapa da disciplina, focaremos na construção da **camada de interface da aplicação**.

Nesta fase, utilizaremos uma **API fake** para simular o comportamento do backen e permitir o desenvolvimento completo da interface e dos fluxos da aplicação.

Stack tecnológica do frontend

Utilizaremos as seguintes tecnologias:

- TypeScript
- Next.js
- Chakra UI
- Tailwind CSS
- Zustand
- API Fake

Escopo esperado da Avaliação 1

Ao final desta etapa, a aplicação deverá apresentar pelo menos:

- Página inicial ou dashboard;
- Listagem de demandas;
- Formulário de criação de demanda;

- Visualização de detalhes da demanda;
- Painel administrativo para gestores;
- Sistemas de Filtros;
- Navegação entre páginas;
- Integração com API fake.
- Menos prioritárias:
  - Tela de cadastro;
  - Tela de login;

Requisitos técnicos considerados na Avaliação 1

Durante a avaliação, observaremos principalmente os seguintes aspectos técnicos.

Estrutura e organização do projeto

Será considerado:

- Organização clara de pastas;
- Separação entre componentes, páginas, serviços e stores;
- Legibilidade do código;
- Padronização de nomenclatura.

Uso de TypeScript

Observaremos:

- Tipagem adequada de dados;
- Definição de interfaces e tipos;
- Consistência na modelagem de estruturas;
- Redução do uso de `any`.

Uso do Next.js

Será considerado:

- Organização adequada das rotas;
- Estruturação das páginas;
- Separação de responsabilidades;
- Organização geral do projeto.

Componentização

Observaremos se a interface foi construída com:

- Componentes reutilizáveis;
- Separação entre layout e lógica;
- Reaproveitamento de código.

Interface e experiência do usuário

Será observado:

- Clareza visual;
- Consistência entre telas;
- Hierarquia de informação;
- Feedback visual de ações;
- Tratamento de estados de carregamento.

Uso de Chakra UI e Tailwind

Será considerado:

- Padronização visual;
- Organização dos estilos;
- Uso coerente das bibliotecas.

Gerenciamento de estado com Zustand

Observaremos:

- Modelagem adequada das stores;
- Uso correto de estado global;
- Organização do fluxo de dados.

Integração com API Fake

Será considerado:

- Consumo correto de endpoints;
- Tratamento de loading;
- Tratamento de erro;
- Coerência no mapeamento de dados.

Formulários

Observaremos:

- Validações básicas;
- Campos coerentes com o domínio;
- Tratamento adequado de erros.

Responsividade

A interface deverá funcionar adequadamente em diferentes tamanhos de tela.

Será considerado:

- Adaptação para desktop e mobile;
- Manutenção da usabilidade;

- Ausencia de quebras graves de layout.

Entregáveis da Avaliação 1

Cada grupo deverá entregar:

- **Repositório do projeto**
- **Deploy da aplicação**
- **Vídeo demonstrando o sistema**

Avaliação 2

Backend + Integração

Na segunda etapa da disciplina, desenvolveremos o **backend da aplicação** e integraremos todas as camadas do sistema.

Stack tecnológica do backend

Utilizaremos as seguintes tecnologias:

- Node.js
- Express
- Prisma ORM
- JWT
- Cookies
- Redis
- Cron Jobs

Escopo esperado da Avaliação 2

Nesta etapa, a aplicação evoluirá para incluir:

- Backend funcional;
- Banco de dados persistente;
- Autenticação real;
- Integração completa entre frontend e backend;
- Implementação das regras de negócio;
- Uso de Redis (quando justificado);
- Uso de cron jobs (quando justificado).

Requisitos técnicos considerados na Avaliação 2

Arquitetura backend

Observaremos:

- Organização do projeto;
- Separação entre rotas, controllers, services e repositórios;
- Clareza de responsabilidades;
- Facilidade de manutenção.

Modelagem de dados com Prisma

Será considerado:

- Modelagem adequada das entidades;
- Relacionamentos entre dados;
- Consistência do schema;
- Uso correto do ORM.

Persistência de dados

Persistência de dados

Observaremos:

- Funcionamento do CRUD principal;
- Consistência entre frontend e banco de dados;
- Integridade dos dados.

API REST

Será considerado:

- Organização dos endpoints;
- Uso correto de métodos HTTP;
- Padronização das respostas;
- Tratamento de erros.

Autenticação

Implementaremos:

- Cadastro de usuários;
- Login de usuários;
- Autenticação com JWT;
- Persistência de sessão com cookies;
- Proteção de rotas privadas;
- Refresh ou persistência de token.

Perfis de usuário

A aplicação deverá permitir **perfis distintos de usuário**, como por exemplo

- Cidadão;
- Gestor público.

Cada perfil poderá ter permissões diferentes dentro da aplicação.

Integração Frontend + Backend

Será considerado:

- Substituição da API fake pela API real;
- Funcionamento completo da aplicação;
- Sincronização correta de dados;
- Tratamento adequado de erros da API.

Redis

O uso de Redis não será **obrigatório**.

Cada equipe deverá:

- Decidir em quais partes do sistema utilizar Redis;
- Justificar tecnicamente essa escolha.

Possíveis usos incluem:

- Cache de consultas frequentes;
- Armazenamento de sessões;
- Otimização de endpoints com alto volume de leitura.

Cron Jobs

O uso de **cron jobs também não será obrigatório**.

Os cron jobs deverão executar alguma tarefa automatizada relevante para o sistema, como por exemplo:

- Cálculo periódico de métricas;
- Consolidação de indicadores;
- Limpeza de dados temporários.

Qualidade do código

Qualidade de código

Observaremos:

- Legibilidade;
- Organização;
- Separação de responsabilidades;
- Redução de duplicação de código;
- Facilidade de manutenção.

Entregáveis da Avaliação 2

Cada grupo deverá entregar:

- **Repositório final do projeto**
- **Deploy do sistema**
- **Vídeo demonstrando o sistema completo**
- **Documentação Da API**

---

11. Estrutura recomendada do projeto

Para facilitar a organização dos projetos e manter consistência entre as equipes, utilizaremos uma **estrutura de pastas sugerida**.

Os grupos podem realizar pequenas adaptações, mas recomenda-se manter uma organização semelhante.

Estrutura geral do repositório

```
projeto-smartcity/ │ ├── frontend/ │ ├── backend/ │ ├── docs/ │ ├──
README.md
```

Estrutura do Frontend

Dentro da pasta **frontend**, utilizaremos uma organização baseada em responsabilidades.

```
frontend/ │ ├── src/ │ │ ├── app/ │ │ ├── login/ │ │ ├── cadastro/
│ │ ├── demandas/ │ │ ├── dashboard/ │ │ └── layout.tsx │ │ ├── com
ponents/ │ │ ├── ui/ │ │ ├── forms/ │ │ ├── layout/ │ │ └── demanda
s/ │ │ ├── services/ │ │ └── api.ts │ │ ├── store/ │ │ ├── authStor
e.ts │ │ └── demandStore.ts │ │ ├── hooks/ │ │ ├── types/ │ │ ├── u
tils/ │ │ └── styles/ │ ├── public/ │ ├── package.json
```

Descrição das principais pastas do fronten

app/

Contém as páginas da aplicação utilizando a estrutura de rotas do Next.js.

Exemplos:

- login
- cadastro
- dashboard
- listagem de demandas
- detalhes da demanda

---

components/

Contém componentes reutilizáveis da interface.

Exemplos:

```
components/ ├── ui/ ├── layout/ ├── forms/ └── demandas/
```

---

services/

Contém funções responsáveis por realizar chamadas para a API.

Exemplo:

```
services/api.ts services/authService.ts services/demandService.ts
```

---

store/

Contém os estados globais da aplicação utilizando **Zustand**.

Exemplo:

```
authStore.ts demandStore.ts
```

---

hooks/

Contém hooks personalizados da aplicação.

Exemplo:

```
useAuth.ts useDemands.ts
```

---

types/

Contém interfaces e tipos TypeScript utilizados na aplicação.

Exemplo:

```
User.ts Demand.ts Auth.ts
```

---

utils/

Funções utilitárias reutilizáveis.

Exemplo:

```
formatDate.ts statusLabel.ts
```

▼ Estrutura do Backend

A estrutura do backend será organizada por camadas.

```
backend/ │ ├── src/ │ │ ├── routes/ │ │ ├── authRoutes.ts │ │ ├── d
emandRoutes.ts │ │ └── metricsRoutes.ts │ │ ├── controllers/ │ │ ├
── authController.ts │ │ ├── demandController.ts │ │ └── metricsCont
roller.ts │ │ ├── services/ │ │ ├── authService.ts │ │ ├── demandSe
rvice.ts │ │ └── metricsService.ts │ │ ├── repositories/ │ │ ├── mi
ddlewares/ │ │ ├── authMiddleware.ts │ │ └── errorMiddleware.ts │ │
├── config/ │ │ ├── redis.ts │ │ └── cron.ts │ │ ├── utils/ │ │ └──
server.ts │ ├── prisma/ │ ├── package.json
```

Descrição das principais pastas do backen

routes/

Define os endpoints da API.

Cada arquivo organiza um grupo de rotas.

---

controllers/

Responsáveis por receber a requisição HTTP e retornar a resposta.

---

services/

Contém a lógica de negócio da aplicação.

---

repositories/

Responsáveis pela comunicação com o banco de dados.

---

middlewares/

Contém middlewares como:

- autenticação
- tratamento de erros
- validação de requisições

---

config/

Configurações de serviços externos.

Exemplo:

- Redis
- Cron Jobs

---

12. Lista recomendada de endpoints da API

A seguir está uma **lista sugerida de endpoints** que podem ser implementados no projeto.

Essa estrutura ajuda a manter consistência entre as equipes.

Autenticação

Criar conta

```
POST /auth/register
```

Login

```
POST /auth/login
```

Logout

```
POST /auth/logout
```

Usuário autenticado

```
GET /auth/me
```

Retorna informações do usuário logado.

Usuários

Listar usuários

```
GET /users
```

Buscar usuário

```
GET /users/:id
```

Demandas Urbanas

Criar demanda

```
POST /demands
```

Listar demandas

```
GET /demands
```

```
```

Pode incluir filtros como:

```
?status= ?categoria= ?regiao=
```

Buscar demanda específica

```
GET /demands/:id
```

Atualizar demanda

```
PUT /demands/:id
```

Atualizar status da demanda

```
PATCH /demands/:id/status
```

Excluir demanda

```
DELETE /demands/:id
```

Métricas

Quantidade total de demandas

```
GET /metrics/total-demands
```

Demandas por categoria

```
GET /metrics/demands-by-category
```

Tempo médio de atendimento

```
GET /metrics/average-response-time
```

Exemplo de rotas protegidas

Algumas rotas devem exigir autenticação.

Exemplo:

```
POST /demands PATCH /demands/:id/status GET /metrics/*
```

Possível divisão de permissões

Exemplo:

Cidadão

- Criar demandas
- Visualizar próprias demandas
- Acompanhar status

Gestor

- Visualizar todas as demandas
- Atualizar status
- Visualizar métricas

Observação importante

A lista de endpoints apresentada é **uma recomendação de organização**, não uma obrigação rígida.

Cada equipe poderá adaptar a estrutura da API, desde que:

- A aplicação funcione corretamente;
- As funcionalidades mínimas sejam atendidas;
- Haja consistência na organização das rotas e regras de negócio.

Considerações finais

Este projeto busca simular um cenário real de desenvolvimento de software.